INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI 3

Rapport du PI Robot

Projet Intégratif Smart Robot

Étudiants

GRONDIN	Yann	yann.grondin@insa-rouen.fr
KAMARA	Mohamed	mohamed.kamara@insa-rouen.fr
FAKHOURY	Zouhair	zouhair.fakhoury@insa-rouen.fr
SAVARY	Théo	theo.savary@insa-rouen.fr
SAAD DJABALLAH	Seif-Din	seif-din.saad_djaballah@insa-rouen.fr

19 décembre 2025

Table des matières

1 Introduction **2**

2 Partie electronique **3**
2.1 Schema electrique 3
2.2 Choix de cablage et decisions techniques 3
2.3 Bilan de puissance et autonomie 3
2.4 Description des composants 3
2.5 Organigramme de la demarche 3
2.6 Difficultes et ameliorations 4

3 Partie informatique **5**
3.1 Analyse 5
3.2 Conception Préliminaire 5
3.3 Conception détaillée et algorithmes 7
3.4 Programme principal du robot 9
3.5 Programme de gestion des intersections 10
3.6 Programme de suivi de ligne 10
3.7 Implémentation et Tests 10

4 Conclusion et matrice de contribution **13**
4.1 Qui a fait quoi 13

1 Introduction

Le projet d'intégration PI Smart Robot consiste à concevoir un robot mobile capable de parcourir le plus vite possible une ville modélisée par une grille de cases reliées par des lignes noires. Le robot est positionné au départ sur une pastille rouge avec une orientation connue (Nord, Est, Sud ou Ouest). Il doit visiter toutes les pastilles vertes en émettant un "bip" à chaque point de passage, puis revenir à son point de départ. La description de la ville est fournie dans un fichier texte (largeur l, case et orientation initiales, liste des passages entre cases, liste des cases à visiter, terminee par un point) qui suit la numérotation donnée. La commande du robot se fait à l'aide d'un flux d'ordres élémentaires `AV` (avancer jusqu'au prochain croisement), `TG` (tourner à gauche) et `TD` (tourner à droite), et s'achève par un point.

Le jour de la démonstration, un script bash lance deux programmes communicants : le premier lit le fichier de description et produit sur sa sortie standard la séquence d'ordres `AV TG TD ...` Le second lit cette séquence, interprète les ordres et pilote le robot. Le robot est comparé aux autres équipes grâce aux temps d'exécution total.

Toutes les équipes disposent des mêmes composants, cependant le choix revient aux équipes sur lesquelles utiliser.

Tout d'abord, nous présenterons la partie électronique, elle détaillera la conception et la réalisation du robot. Puis, nous verrons la partie informatique qui poursuivra sur la mise en oeuvre d'un cycle en V ainsi que les algorithmes non triviaux utilisés. Enfin, nous conclurons par une synthèse des contributions individuelles, des perspectives d'amélioration et les difficultés rencontrées.

2 Partie electronique

Presenter la conception et la realisation electronique du robot.

2.1 Schema electrique

Inserer le schema complet (Raspberry Pi, L293D, capteurs, batteries, interrupteur, buzzer, LED, ecran).

2.2 Choix de cablage et decisions techniques

Justifier les branchements entre la Raspberry Pi, le controleur moteur, les capteurs et l'alimentation.

- Organisation des masses communes et protections (diodes de roue libre, resistances eventuelles).
- Distribution d'alimentation : batteries NIMH pour la puissance moteur, batterie USB-C pour la Raspberry Pi, logique du bouton on/off.
- Raccordement des capteurs de ligne et du capteur couleur (interfaces, calibrations prevues).
- Choix des broches GPIO utilisees et justification (PWM, interruptions, I2C/SPI).

2.3 Bilan de puissance et autonomie

Completer les valeurs mesurees et l'autonomie estimee.

Sous-ensemble	Tension	Courant	Commentaires
Raspberry Pi	...	...	Charge moyenne (CPU/GPIO)
Pont en H + moteurs	...	...	Courant en regime nominal / de-blocage
Capteurs (ligne, couleur)	...	...	Inclure ecran, LED, buzzer
Total	...	...	**Autonomie estimee : ...**

Table 1: Bilan de puissance et estimation d'autonomie.

2.4 Description des composants

Expliquer le role de chaque composant et son principe de fonctionnement.

- Capteurs de ligne et algorithme de recherche de ligne (role du capteur central vs lateraux).
- Capteur couleur et logique de detection des pastilles (vert/bleu) avec bip et affichage ecran.
- Controleur L293D et commande des moteurs (PWM, direction, freins eventuels).
- Interrupteur, LED et buzzer pour l'interface utilisateur.

2.5 Organigramme de la demarche

Fournir un organigramme illustrant la logique generale (suivi de ligne, detection, gestion des intersections, multithread/interruption si utilisee).

Organigramme a ajouter ici : `figures/organigramme_elec.pdf`.

Figure 1: Organigramme de la boucle principale et des interruptions.

2.6 Difficultes et ameliorations

Lister les problemes rencontres (bruit capteur, couple moteur, poids) et les pistes d'amelioration.

3 Partie informatique

Cette seconde partie décrit la démarche logicielle adoptée dans le cadre du cycle en V : analyse du problème, conception des modules, choix des algorithmes, puis implémentation et tests. Deux programmes communicants sont réalisés : un planificateur qui lit le fichier texte de description de la ville et produit une suite d'ordres, un pilote qui lit ces ordres sur son entrée standard et contrôle le robot en temps réel tout en affichant les informations sur l'écran LCD et en émettant un signal sonore sur les pastilles.

3.1 Analyse

3.1.1 Analyse Descendante

L'architecture du projet a été conçue selon une approche descendante (Top-Down) pour isoler les responsabilités et faciliter les tests. Le système est décomposé en niveaux hiérarchiques, comme illustré par le diagramme suivant qui reprend l'ensemble des fonctions implémentées.

Le diagramme d'analyse descendante pour la partie planification en annexe.

Les niveaux sont :

- **Niveau orchestration (Niveau 1)** : Programme principal et gestion du cycle de vie.
- **Niveau métier (Niveau 2)** : Algorithmes de chargement (Parsing) et de planification (TSP).
- **Niveau noyau (Niveau 3)** : Algorithme de recherche de chemin A*.
- **Niveau données (Niveau 4)** : Opérations élémentaires sur les Types Abstraits de Données (TAD).

3.2 Conception Préliminaire

Cette phase définit les TAD utilisés pour modéliser le problème, en correspondance directe avec l'implémentation C.

3.2.1 TAD Case

Correspondance code C : `unsigned int` (identifiant de noeud).

Paramètres

- `id : Naturel`

Utilise

- Entier

Opérations

- `creerCase : Entier → Case`
- `estEgal : Case × Case → Booléen`
- `valeur : Case → Entier`

Axiomes

- `valeur(creerCase(i)) = i`
- `estEgal(c1, c2) = (valeur(c1) == valeur(c2))`

3.2.2 TAD Direction

Correspondance code C : `enum Direction { Nord=0, Est=1, Sud=2, Ouest=3 }`.

Paramètres

- `dir : Entier` (0..3)

Utilise

- Entier, Booléen

Opérations

- `Nord, Est, Sud, Ouest : → Direction`
- `tournerDroite : Direction → Direction`
- `tournerGauche : Direction → Direction`
- `opposee : Direction → Direction`
- `directionVersEntier : Direction → Entier`
- `entierVersDirection : Entier ⇸ Direction`

Axiomes

- `tournerDroite(Nord) = Est`, `tournerDroite(Est) = Sud`, `tournerDroite(Sud) = Ouest`, `tournerDroite(Ouest) = Nord`
- `tournerGauche(d)` = `opposee(tournerDroite(d))`
- `opposee(Nord) = Sud`

3.2.3 TAD Labyrinthe

Correspondance code C : Structure `CityConfig` contenant un `G_Graphe`.

Paramètres

- `largeur, hauteur : Entier`
- `graphe : Graphe`
- `checkpoints : Liste<Case>`
- `depart : Case`, `directionDepart : Direction`

Utilise

- Entier, Case, Direction, Graphe, Liste

Opérations

- `creerLabyrinthe : Entier × Entier × Graphe × ... → Labyrinthe`
- `obtenirLargeur, obtenirHauteur : Labyrinthe → Entier`
- `estConnecte : Labyrinthe × Case × Case → Booléen`
- `caseVersCoord : Labyrinthe × Case → (x:Entier, y:Entier)`
- `coordVersCase : Labyrinthe × Entier × Entier ⇸ Case`

Axiomes

- `estConnecte(l, c1, c2) = G_arcPresent(l.graphe, c1, c2)`
- `caseVersCoord : ID = y * largeur + x + 1`

3.2.4 TAD Instruction (Action)

Correspondance code C : `enum ActionType { AV=0, TD=1, TG=2 }`.

Opérations

- `AV, TD, TG : → ActionType`

3.2.5 TAD ListeInstructions (Chemin)

Correspondance code C : Tableau de `Step` ou Liste Chaînée.

Utilise

- Instruction, Case, Entier

Opérations

- `creerListeVide : → ListeInstructions`
- `ajouterEnFin : ListeInstructions × Instruction × Case → ListeInstructions`
- `concatener : ListeInstructions × ListeInstructions → ListeInstructions`
- `longueur : ListeInstructions → Entier`

Axiomes

- `longueur(concatener(l1, l2)) = longueur(l1) + longueur(l2)`

3.2.6 Partie Robot

Pour le robot, nous n'avons pas de TAD. Cependant, nous avons réalisé une analyse descendante des fonctions et procédures requises pour la gestion du robot. Nous allons seulement aller en détail sur la gestion des instructions car c'est la partie logique du robot. Le schéma se trouver en annexe de ce rapport. L'ensemble des pseudocodes des fonctions est dans `programmes/utils/pseudocodes_repartition.md` sur notre dépôt GitLab. Nous présenterons tout de même le programme principal, la gestion des intersections pour la partie robot et le suivi de ligne.

3.3 Conception détaillée et algorithmes

Cette section détaille les algorithmes majeurs (Intelligence et Logique du robot).

3.3.1 Algorithme A* (Espace d'Etats)

Optimisation clé est la recherche dans l'espace $(Noeud, Direction)$ pour inclure le coût des rotations.

```
Structure State
    node: Entier
    dir: Direction
FinStructure
```

```
Fonction astar_search(graphe, start_node, start_dir, target_node) : Liste<Action>
    OpenSet <- FilePriorite()
    ClosedSet <- Tableau[NOEUDS][4] initialise a INFINI

    start_state <- State(start_node, start_dir)
    OpenSet.ajouter(NodeData(start_state, 0, heuristique(start_node, target_node)))
    ClosedSet[start_node][start_dir] <- 0

    Tant que non OpenSet.estVide() faire
        current <- OpenSet.extraireMin()
        // Si on atteint le noeud cible (peu importe la direction finale)
        Si current.state.node == target_node alors
            Retourner reconstruire_chemin(current)
        FinSi

        // 1. Action AVANCER
        voisin <- get_voisin_devant(graphe, current.state.node, current.state.dir)
        Si voisin existe alors
            new_g <- current.g_score + COST_MOVE
            Si new_g < ClosedSet[voisin][current.state.dir] alors
                ClosedSet[voisin][current.state.dir] <- new_g
                next_state <- State(voisin, current.state.dir)
                OpenSet.ajouter(NodeData(next_state, new_g, new_g + h(voisin), current.state, AV))
            FinSi
        FinSi

        // 2. Action TOURNER DROITE
        dir_droite <- (current.state.dir + 1) mod 4
        new_g_rot <- current.g_score + COST_TURN
        Si new_g_rot < ClosedSet[current.state.node][dir_droite] alors
             ClosedSet[current.state.node][dir_droite] <- new_g_rot
             next_state <- State(current.state.node, dir_droite)
             OpenSet.ajouter(NodeData(next_state, new_g_rot,new_g_rot+h(current.state.node),current.state,TD))
        FinSi

        // 3. Action TOURNER GAUCHE
        dir_gauche <- (current.state.dir + 3) mod 4
        new_g_gauche <- current.g_score + COST_TURN
        Si new_g_gauche < ClosedSet[current.state.node][dir_gauche] alors
            ClosedSet[current.state.node][dir_gauche] <- new_g_gauche
            next_state <- State(current.state.node, dir_gauche)
            OpenSet.ajouter(NodeData(
                next_state,
                new_g_gauche,
                new_g_gauche+h(current.state.node),
                current.state,TG)
            )
        FinSi
    FinTantQue
    Retourner ECHEC
Fin
```

3.3.2 Voyageur de Commerce (TSP) Hybride

On utilise une stratégie hybride pour visiter les checkpoints de manière optimale.

```
Fonction solve_tsp(graphe, start_node, start_dir, checkpoints) : Liste<Action>
    Si longueur(checkpoints) <= 10 alors
        // Approche Exhaustive
        best_cost <- INFINI
        best_path <- VIDE
        Pour chaque permutation P de checkpoints faire
            (cost, path) <- evaluer_sequence(start_node, start_dir, P)
            Si cost < best_cost alors
                best_cost <- cost
                best_path <- path
            FinSi
        FinPour
        Retourner best_path
    Sinon
        // Approche Gloutonne
```

```
        curr_node <- start_node
        curr_dir <- start_dir
        path_total <- Vide
        Tant que checkpoints non visites faire
            best_next_cp <- trouver_plus_proche_astar(curr_node, curr_dir, checkpoints)
            segment <- astar_search(graphe, curr_node, curr_dir, best_next_cp)
            path_total += segment
            (curr_node, curr_dir) <- fin(segment)
            Retirer best_next_cp de checkpoints
        FinTantQue
        // Retour au depart
        path_total += astar_search(graphe, curr_node, curr_dir, start_node)
        Retourner path_total
    FinSi
Fin
```

3.3.3 Parsing du Labyrinthe

Chargement de la configuration depuis un fichier texte.

```
Fonction load_city(filename) : CityConfig
    config.graphe <- NouveauGraphe()

    Lire N (taille)
    Lire StartNode, StartDir

    Tant que non EOF faire
        Si ligne est "U-V" alors
            AjouterArete(config.graphe, U, V)
        Sinon Si ligne est "N" alors
            AjouterCheckpoint(config.checkpoints, N)
        FinSi
    FinTantQue

    Retourner config
Fin
```

3.4 Programme principal du robot

```
fonction principale main() : entier
    debut
        debut_temps <- temps_actuel_millis()

        setupPin()
        lcd_display_init()

        nb_instr <- 0

        creer_thread(thread_RGB, rgb_color_thread_loop, NULL)

        instructions_robot <- charger_instructions_dynamique(stdin, nb_instr)
        ecrire("J'ai recu " + nb_instr + " instructions.")

        pour i de 0 a nb_instr-1 faire
            ecrire("Instruction " + i + " : " + instructions_robot[i])

            si instructions_robot[i] = "AV" alors
                instr <- 0
                afficher_LCD(1,"Avancer")
            sinon si instructions_robot[i] = "TG" alors
                instr <- 1
                afficher_LCD(1,"Gauche")
            sinon si instructions_robot[i] = "TD" alors
                instr <- 2
                afficher_LCD(1,"Droite")
            sinon
                ecrire("Instruction inconnue: " + instructions_robot[i])
                stop()
```

```
                continuer
            fin si

            gestionIntersection(instr)
        fin pour

        // Liberation memoire
        pour i de 0 a nb_instr-1 faire
            liberer(instructions_robot[i])
        fin pour
        liberer(instructions_robot)

        afficher_LCD(0,"Circuit")

        fin_temps <- temps_actuel_millis()
        duree <- (fin_temps - debut_temps)/1000.0  // conversion en secondes
        afficher_LCD(1, convertir_en_chaine(duree) + " s")

        ecrire("Duree totale : " + convertir_en_chaine(duree) + " secondes")

        attendre_thread(thread_RGB)
        stop()

        retourner 0
    fin
```

3.5 Programme de gestion des intersections

```
procedure gestionIntersection(instr : entier)
    debut
        selon instr faire
            cas 0 : gestionSuiviLigne(DELAIS_INTERSECTION)
            cas 1 : gaucheIntersection(DELAIS_INTERSECTION)
            cas 2 : droiteIntersection(DELAIS_INTERSECTION)
            autre : ecrire("Instruction invalide")
        fin selon
    fin
```

3.6 Programme de suivi de ligne

```
procedure gestionSuiviLigne(delais : entier)
    debut
        avance()
        tant que NON surIntersection() faire
            si lecturePin(CAPTEUR_LIGNE_GAUCHE) ET NON lecturePin(CAPTEUR_LIGNE_MILIEU) alors
                gauche()
            sinon si lecturePin(CAPTEUR_LIGNE_DROIT) ET NON lecturePin(CAPTEUR_LIGNE_MILIEU) alors
                droite()
            sinon
                avance()
            fin si
            attendre(delais)
        fin tant que
    fin
```

3.7 Implémentation et Tests

L'implémentation respecte la structure standard C (src/include).

3.7.1 Tests Unitaires

Une suite de tests unitaires a été développée à l'aide du framework **CUnit**. Ces tests sont situés dans le répertoire `programmes/src/algo` et se basent sur les axiomes définis dans

les spécifications des TADs.

Les tests couvrent les modules suivants :

- **TAD Labyrinthe** (`test_labyrinthe.c`) : Vérifie le chargement correct des dimensions, la position initiale, et la connectivité du graphe (fonction `estConnecte`). Il valide également les axiomes de conversion de coordonnées (`caseVersCoord`).
- **TAD Instructions** (`test_instructions.c`) : Valide la structure des données `Step` et les énumérations `ActionType` et `Direction`.
- **Graphe** (`test_graphe.c`) : Teste les opérations de base du graphe générique : création, ajout et suppression de sommets et d'arcs, ainsi que la récupération des sommets adjacents.
- **Algorithme A*** (`test_astar.c`) : Valide la recherche de chemin sur des graphes simples (grilles) et vérifie le comportement de l'algorithme dans des cas limites (chemin inexistant).

3.7.2 Exemple de Validation

Pour valider le fonctionnement de l'algorithme de planification sur un cas concret, nous présentons ici un exemple d'exécution complet. Cette validation permet de confirmer que le chemin généré respecte bien les contraintes du labyrinthe (murs, sens de rotation) et visite tous les checkpoints demandés.

Espace de test (Fichier d'entrée) Le fichier descriptive ci-dessous définit une grille de 5x5 avec des murs et 3 checkpoints à visiter (13, 22, 20).

```
5
6S
1-2
1-6
2-3
2-7
3-4
4-5
5-10
6-11
7-8
8-13
9-14
10-15
11-16
11-12
12-17
12-13
14-19
15-20
16-21
17-22
18-19
18-23
19-20
20-25
21-22
22-23
23-24
13
22
20
.
```

Résultat (Instructions générées) Le planificateur produit la séquence d'instructions suivante, qui mène le robot successivement aux points 13, 22, et 20 avant de revenir au départ.

```
AV
TG
AV
AV
AV
TD
AV
TG
AV
TD
AV
TD
AV
AV
AV
TD
AV
AV
.
```

Visualisation La figure ci-dessous illustre graphiquement le chemin calculé par l'algorithme au sein du labyrinthe, confirmant le passage par les points d'intérêt.

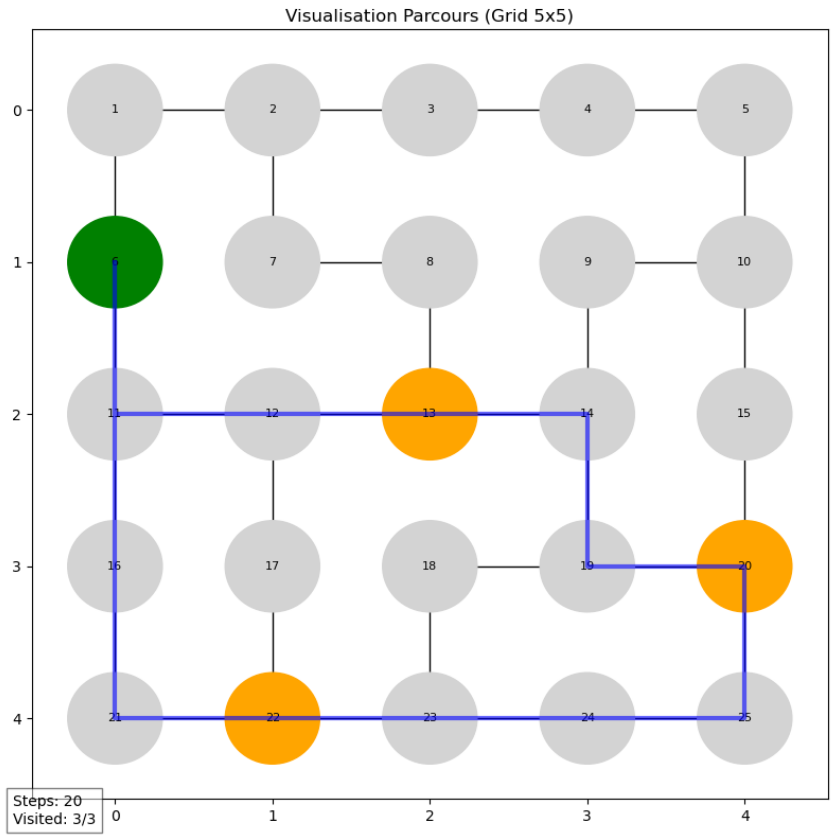


Figure 2: Visualisation du chemin calculé pour l'exemple de validation

4 Conclusion et matrice de contribution

Conclure sur les performances obtenues et les enseignements.

- Synthese des resultats (temps realise, fiabilite du parcours, competitions).
- Retours d'experience et perspectives d'amelioration.

4.1 Qui a fait quoi

Completer la matrice des contributions.

Membre	Electronique	Informatique	Tests/Integration	Documentation
Nom 1	...	...	...	...
Nom 2	...	...	...	...
Nom 3	...	...	...	...
Nom 4	...	...	...	...
Nom 5	...	...	...	...

Table 2: Matrice de contribution des membres du projet.